

buscar_coords_antena

September 30, 2025

```
[9]: import pandas as pd
import requests
import os
import time
from dotenv import load_dotenv
from tqdm import tqdm

[10]: # --- CONFIGURACIÓN ---
# Carga las variables de entorno (mi API_TOKEN en el archivo .env)
load_dotenv()
API_TOKEN = os.getenv("AQI_API_KEY")
BASE_URL = "https://api.waqi.info"

[11]: # Archivo de entrada (el que limpié manualmente)
archivo_entrada = 'lista_antenas.csv'
# Archivo de salida donde se guardarán los resultados
archivo_salida = 'antenas_con_coordenadas.csv'
# -----

[12]: # Verificamos que el token de la API se haya cargado
if not API_TOKEN:
    print(" Error: No se encontró el API_TOKEN. Asegúrate de que tu archivo .
    ↪env está en la misma carpeta y tiene la variable API_TOKEN.")
else:
    try:
        # Leemos el archivo con la lista de antenas limpias
        df_antenas = pd.read_csv(archivo_entrada)

        # Lista para ir guardando los resultados de cada antena
        resultados_finales = []

        print(f"Iniciando la búsqueda de coordenadas para {len(df_antenas)}
        ↪antenas...")
        print("Esto puede tardar varios minutos...")

        # tqdm nos envuelve el bucle para mostrarnos una barra de progreso
        for nombre_antena in tqdm(df_antenas['antena']):
            lat, lon = None, None # Inicializamos las coordenadas como nulas
```

```

    try:
        # Paso 1: Usar el endpoint de búsqueda de la API
        url_busqueda = f"{BASE_URL}/search/?
↪token={API_TOKEN}&keyword={nombre_antena}"
        response_busqueda = requests.get(url_busqueda)
        response_busqueda.raise_for_status() # Lanza un error si la
↪petición falla

        data_busqueda = response_busqueda.json()

        # Si la API encuentra resultados para esa búsqueda
        if data_busqueda.get("status") == "ok" and data_busqueda.
↪get("data"):
            # Tomamos el identificador del primer resultado (el más
↪probable)
            station_id = data_busqueda["data"][0]["station"]["url"]

            # Paso 2: Con el identificador, obtenemos los datos
↪completos de la estación
            url_feed = f"{BASE_URL}/feed/{station_id}/?
↪token={API_TOKEN}"
            response_feed = requests.get(url_feed)
            response_feed.raise_for_status()

            data_feed = response_feed.json()

            if data_feed.get("status") == "ok":
                # ¡Éxito! Extraemos la latitud y longitud
                coords = data_feed["data"]["city"]["geo"]
                lat, lon = coords[0], coords[1]

    except requests.exceptions.RequestException as e:
        # Si hay un error de red, lo imprimimos pero continuamos
        print(f"\nError de red buscando '{nombre_antena}': {e}")

    # Agregamos el resultado (con o sin coordenadas) a nuestra lista
    resultados_finales.append({
        'antena': nombre_antena,
        'latitud': lat,
        'longitud': lon
    })

    # Pausa de 0.5 segundos entre cada petición para no sobrecargar la
↪API
    time.sleep(0.5)

```

```

# Convertimos la lista de resultados en un DataFrame de Pandas
df_final = pd.DataFrame(resultados_finales)

# Guardamos el DataFrame en el nuevo archivo CSV
df_final.to_csv(archivo_salida, index=False)

print(f"\n Proceso completado, Se ha creado el archivo_
↪ '{archivo_salida}'.")
print("Algunas pueden estar vacías si la API no encontró la antenna.")

except FileNotFoundError:
    print(f" Error: No se encontró el archivo de entrada_
↪ '{archivo_entrada}'. Asegúrate de que el nombre sea correcto.")

```

Iniciando la búsqueda de coordenadas para 86 antenas...
 Esto puede tardar varios minutos...

0%	0/86 [00:00<?,
?it/s]	
1%	1/86 [00:02<03:22,
2.38s/it]	
2%	2/86 [00:04<03:29,
2.50s/it]	
3%	3/86 [00:07<03:29,
2.52s/it]	
5%	4/86 [00:10<03:28,
2.54s/it]	
6%	5/86 [00:12<03:32,
2.62s/it]	
7%	6/86 [00:15<03:27,
2.59s/it]	
8%	7/86 [00:17<03:24,
2.59s/it]	
9%	8/86 [00:20<03:12,
2.47s/it]	
10%	9/86 [00:22<03:10,
2.48s/it]	
12%	10/86 [00:26<03:36,
2.85s/it]	
13%	11/86 [00:28<03:27,
2.76s/it]	
14%	12/86 [00:31<03:24,
2.76s/it]	
15%	13/86 [00:33<03:11,
2.63s/it]	
16%	14/86 [00:36<03:06,

2.59s/it]	
17%	15/86 [00:39<03:05,
2.61s/it]	
19%	16/86 [00:41<03:06,
2.66s/it]	
20%	17/86 [00:44<03:09,
2.75s/it]	
21%	18/86 [00:47<03:03,
2.69s/it]	
22%	19/86 [00:50<02:59,
2.68s/it]	
23%	20/86 [00:53<03:10,
2.89s/it]	
24%	21/86 [00:58<03:39,
3.38s/it]	
26%	22/86 [01:00<03:19,
3.12s/it]	
27%	23/86 [01:03<03:04,
2.93s/it]	
28%	24/86 [01:05<02:46,
2.68s/it]	
29%	25/86 [01:07<02:38,
2.60s/it]	
30%	26/86 [01:10<02:35,
2.59s/it]	
31%	27/86 [01:13<02:54,
2.95s/it]	
33%	28/86 [01:16<02:44,
2.83s/it]	
34%	29/86 [01:19<02:47,
2.95s/it]	
35%	30/86 [01:22<02:39,
2.84s/it]	
36%	31/86 [01:26<02:57,
3.23s/it]	
37%	32/86 [01:30<03:08,
3.49s/it]	
38%	33/86 [01:34<03:08,
3.56s/it]	
40%	34/86 [01:36<02:49,
3.26s/it]	
41%	35/86 [01:39<02:39,
3.13s/it]	
42%	36/86 [01:42<02:27,
2.96s/it]	
43%	37/86 [01:45<02:23,
2.93s/it]	
44%	38/86 [01:47<02:16,

2.85s/it]	
45%	39/86 [01:50<02:12,
2.83s/it]	
47%	40/86 [01:52<02:06,
2.75s/it]	
48%	41/86 [01:55<02:02,
2.72s/it]	
49%	42/86 [01:58<02:00,
2.74s/it]	
50%	43/86 [02:01<01:59,
2.77s/it]	
51%	44/86 [02:03<01:53,
2.71s/it]	
52%	45/86 [02:06<01:47,
2.63s/it]	
53%	46/86 [02:08<01:42,
2.57s/it]	
55%	47/86 [02:11<01:41,
2.61s/it]	
56%	48/86 [02:14<01:39,
2.63s/it]	
57%	49/86 [02:16<01:37,
2.62s/it]	
58%	50/86 [02:19<01:31,
2.56s/it]	
59%	51/86 [02:21<01:28,
2.53s/it]	
60%	52/86 [02:24<01:27,
2.57s/it]	
62%	53/86 [02:27<01:27,
2.65s/it]	
63%	54/86 [02:29<01:23,
2.60s/it]	
64%	55/86 [02:32<01:19,
2.56s/it]	
65%	56/86 [02:35<01:28,
2.95s/it]	
66%	57/86 [02:40<01:36,
3.32s/it]	
67%	58/86 [02:47<02:11,
4.69s/it]	
69%	59/86 [02:50<01:52,
4.16s/it]	
70%	60/86 [02:59<02:20,
5.41s/it]	
71%	61/86 [03:01<01:50,
4.42s/it]	
72%	62/86 [03:03<01:29,

3.73s/it]	
73%	63/86 [03:05<01:13,
3.21s/it]	
74%	64/86 [03:09<01:14,
3.37s/it]	
76%	65/86 [03:12<01:12,
3.47s/it]	
77%	66/86 [03:15<01:07,
3.35s/it]	
78%	67/86 [03:18<00:58,
3.10s/it]	
79%	68/86 [03:21<00:55,
3.07s/it]	
80%	69/86 [03:23<00:48,
2.83s/it]	
81%	70/86 [03:26<00:44,
2.75s/it]	
83%	71/86 [03:28<00:40,
2.69s/it]	
84%	72/86 [03:31<00:36,
2.62s/it]	
85%	73/86 [03:34<00:34,
2.68s/it]	
86%	74/86 [03:36<00:31,
2.66s/it]	
87%	75/86 [03:39<00:28,
2.63s/it]	
88%	76/86 [03:42<00:26,
2.67s/it]	
90%	77/86 [03:44<00:23,
2.58s/it]	
91%	78/86 [03:46<00:20,
2.57s/it]	
92%	79/86 [03:49<00:17,
2.51s/it]	
93%	80/86 [03:51<00:15,
2.52s/it]	
94%	81/86 [03:54<00:13,
2.60s/it]	
95%	82/86 [03:57<00:10,
2.64s/it]	
97%	83/86 [04:00<00:07,
2.65s/it]	
98%	84/86 [04:02<00:05,
2.63s/it]	
99%	85/86 [04:05<00:02,
2.60s/it]	
100%	86/86 [04:07<00:00,

2.88s/it]

Proceso completado, Se ha creado el archivo 'antenas_con_coordenadas.csv'.
Algunas pueden estar vacías si la API no encontró la antena.

[]: